

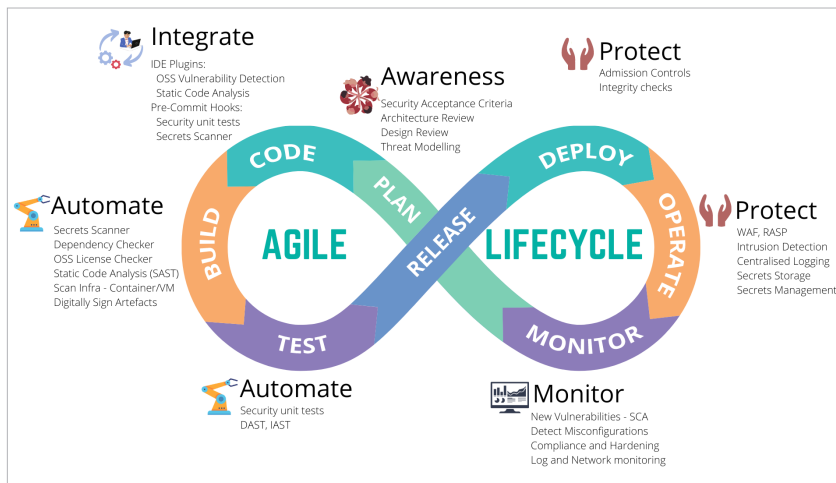


Figure 6: DevSecOps—the complete picture

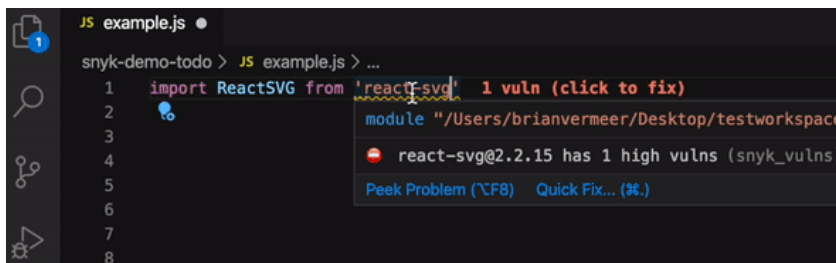


Figure 7: Snyk IDE plugin — detects known vulnerabilities while you code

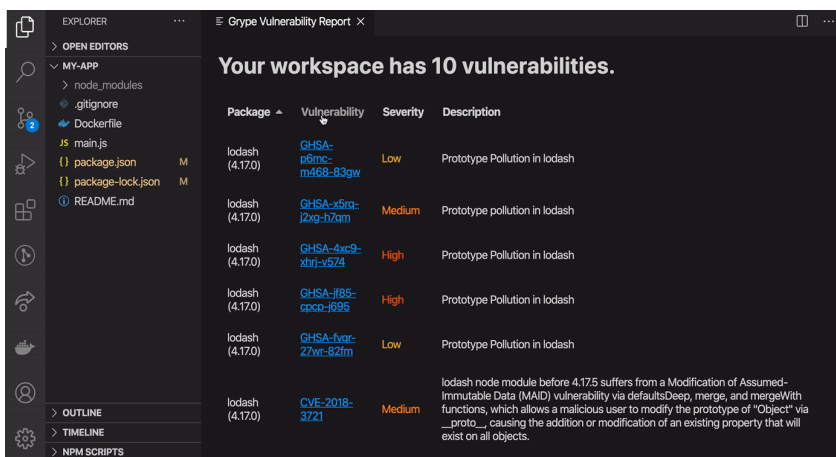


Figure 8: Gripe plugin — detects known vulnerabilities in the code base

There are several plugins available for this. Figure 7 illustrates the Snyk plugin.

The Snyk plugin for Node.js based projects provides suggestions about whether the library used has any known vulnerabilities or not. This provides faster feedback to the user.

The Gripe plugin (Figure 8) scans for vulnerabilities in the libraries in

the background — it scans whenever the list of packages used is modified. Similar to the Snyk plugin, it also gives prompt feedback to the developer.

Automate: Security in CI/CD pipelines

Build pipelines are the sweet spot to automate security using tools.

Here, each and every commit can be scanned for security vulnerabilities or misconfiguration. Security tools have evolved over time to detect several bug patterns that might get missed during manual security reviews.

The following things can be automated in the build pipeline:

- Identification of known vulnerabilities in the application dependencies or libraries
- Identification of known vulnerabilities in the container images or VM images
- Identification of OSS licence compliance violations
- Identification of missing security configurations (secure defaults)
- Identification of missing compliance and security hardenings
- Identification of vulnerabilities in the application code -- static code analysis
- Identification of runtime vulnerabilities -- DAST

Since this is quite a vast topic to cover, we will discuss how we can automate all of the above in my next article.

Protect: Prevent attacks from happening at runtime

After deployment, the application needs to be protected from any unknown threats that could occur. Even if we follow all the above practices, we cannot guarantee that all the security issues are detected. The most common runtime application security protection options are:

- Web application firewall (WAF)
- DDoS protection and rate limiting
- Interactive application security testing (IAST)
- Runtime application self-protection (RASP)

Monitor: Continuously monitor applications and infrastructure

After deployment, the application needs to be monitored to identify security vulnerabilities and misconfigurations